

Infosys Internship 4.0 Project

Documentation

Title

Project Documentation: Tea Leaf Disease Detection

Introduction:

The Tea Leaf Disease Detection project aims to leverage AI to identify diseases in tea leaves and other plants. The project utilizes a Convolutional Neural Network (CNN) model to accurately classify plant diseases, providing a valuable tool for early detection and management. This initiative is significant as it can help reduce crop losses and improve agricultural productivity. The project is developed by Ms. Aishwarya Bazzurla under the guidance of Mrs. Madhu Priya and Mr. Deepak.

Project Scope:

The project encompasses the development of an AI model capable of detecting diseases in tea leaves and other plants using image classification. It includes data preprocessing, model training, integration with a Flask web application, and deployment. The scope does not cover real-time image capture or integration with external databases. Key constraints considered were computational resources and the quality of available datasets.

Requirements:

Functional Requirements:

- Image preprocessing and augmentation
- Model training using CNN (DenseNet)
- Integration with Flask for web application
- Prediction and display of disease classification

Non-Functional Requirements:

- High accuracy and efficiency
- User-friendly web interface
- Scalability for additional plant types
- Maintainability of the codebase

User Stories / Use Cases:

1. As a user, I want to upload an image of a plant leaf to detect any potential disease.
2. As a user, I want to view the classification result and suggested treatments.

Technical Stack:

- **Programming Languages:** Python, HTML, CSS
- **Frameworks/Libraries:** TensorFlow, Keras, NumPy, Pandas, Matplotlib, Flask, Seaborn, OpenCV
- **Databases:** None
- **Tools/Platforms:** A100 GPU on Google Colab

Architecture/Design:

The system architecture comprises several components:

1. Data Preprocessing: Resizing, normalization, and augmentation of input images.
2. Model Training: Using DenseNet architecture to train on processed datasets.
3. Web Application: Flask framework for integrating the trained model and HTML interface.
4. Prediction Module: Processing user-uploaded images and displaying results.

Design Decisions:

- Chose DenseNet for its efficient feature extraction and high accuracy.
- Used Flask for its simplicity and suitability for integrating machine learning models with web interfaces.

Development:

Technologies and Frameworks:

- Utilized Python and libraries like TensorFlow and Keras for model development.
- Flask was used for web application development.
- OpenCV was employed for image processing tasks.

Coding Standards and Best Practices:

- Followed PEP 8 guidelines for Python code.
- Implemented modular and reusable code.
- Conducted code reviews and used version control with Git.

Challenges:

- Handling large datasets and ensuring efficient preprocessing.
- Integrating the model with the Flask application and ensuring smooth predictions.
- Addressed by optimizing data pipelines and using robust error handling.

Testing:

Testing Approach:

- Conducted unit tests for model predictions and HTML templates individually.
- Performed integration tests to ensure seamless interaction between template and model.
- System tests to validate the overall functionality of the application.

Testing Results:

- Identified and resolved bugs related to image preprocessing and model integration.
- Ensured the application meets functional and non-functional requirements.

Deployment:

Deployment Process:

- Deployed the Flask application on a local server for development testing.
- Used Google Colab with A100 GPU for model training due to its powerful deep learning capabilities.
- Included deployment scripts for setting up the application environment.

Deployment Instructions:

1. Set up a Python environment with required libraries.
2. Run the Flask application server.
3. Access the web application through the provided local or network URL.

User Guide:

Using the Application:

1. Upload an image of a plant leaf through the web interface.
2. Click the 'Predict' button to view the classification result.
3. Follow the suggested treatment guidelines displayed.

Troubleshooting Tips:

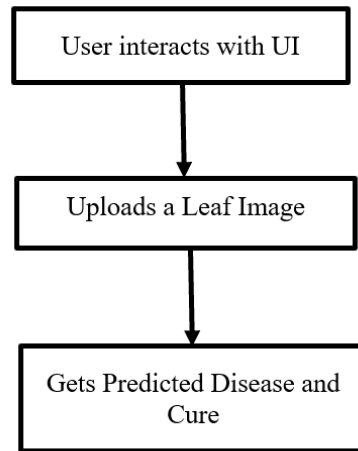
- Ensure the image is clear and well-lit for accurate predictions.
- Check console logs for any error messages if the application fails to run.

Conclusion:

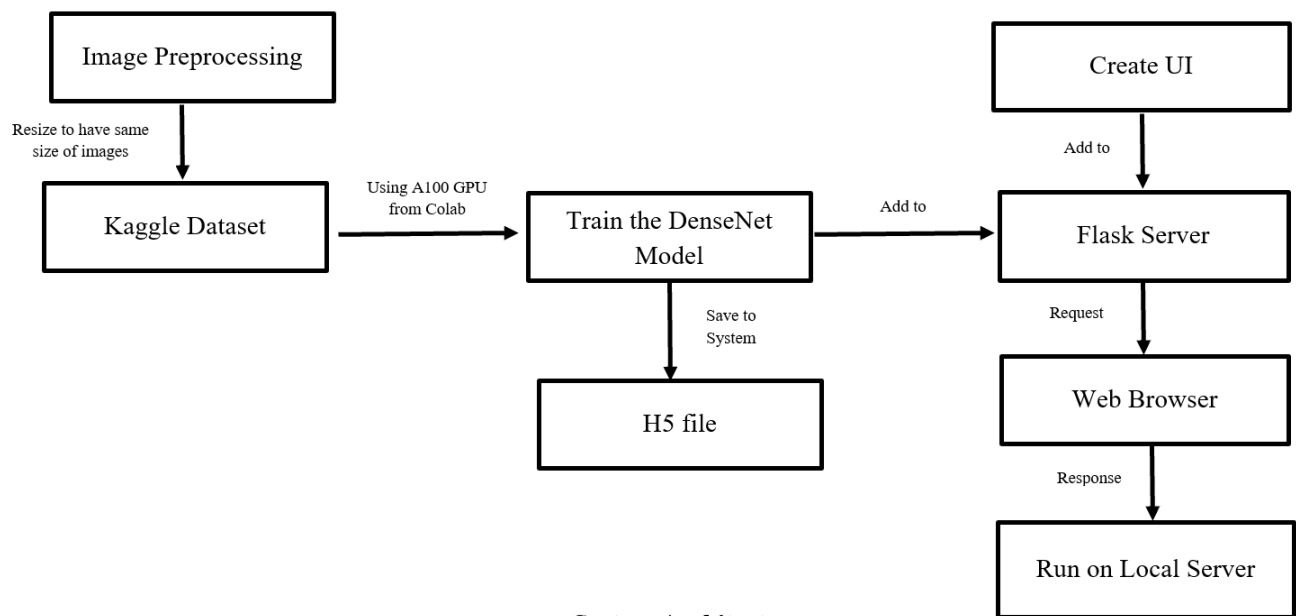
The Tea Leaf Disease Detection project successfully developed a robust AI model and integrated it with a user-friendly web application. The project demonstrated significant potential in improving agricultural practices through early disease detection. Lessons learned include the importance of data quality and the need for scalable solutions. Future improvements could involve real-time image capture and integration with larger agricultural databases.

Appendices:

- **Diagrams:** UML diagrams and system architecture visuals.



Use Case Diagram



System Architecture

- **Code Snippets:** Sample code for data preprocessing, model integration and output.

```
1 > def get_images_avg_width_height(df): ...
22
23 def resize_images_to_avg_dimensions(df, avg_width, avg_height):
24     resized_images = []
25     for i in range(len(df)):
26         img_path = df['image_path'].loc[i]
27         if not img_path.lower().endswith(('.png', '.jpg', '.jpeg', '.bmp', '.tiff', '.tif')):
28             print(f"Skipping non-image file at path {img_path}")
29             continue
30         img = cv2.imread(img_path)
31         if img is None:
32             print(f"Warning: Unable to load image at path {img_path}")
33             continue
34         resized_img = cv2.resize(img, (avg_width, avg_height), interpolation=cv2.INTER_AREA)
35         resized_images.append(resized_img)
36     return resized_images
37
38 avg_width_test, avg_height_test, valid_images_count = get_images_avg_width_height(test_df)
39 print(f"Average width and height for testing data is (avg_width_test)x(avg_height_test) with aspect_ratio (avg_width_test / avg_height_test)")
40
41 resized_images = resize_images_to_avg_dimensions(test_df, avg_width_test, avg_height_test)
[23]
Python
... Average width and height for testing data is 256x255 with aspect_ratio 1.003921568627451
```

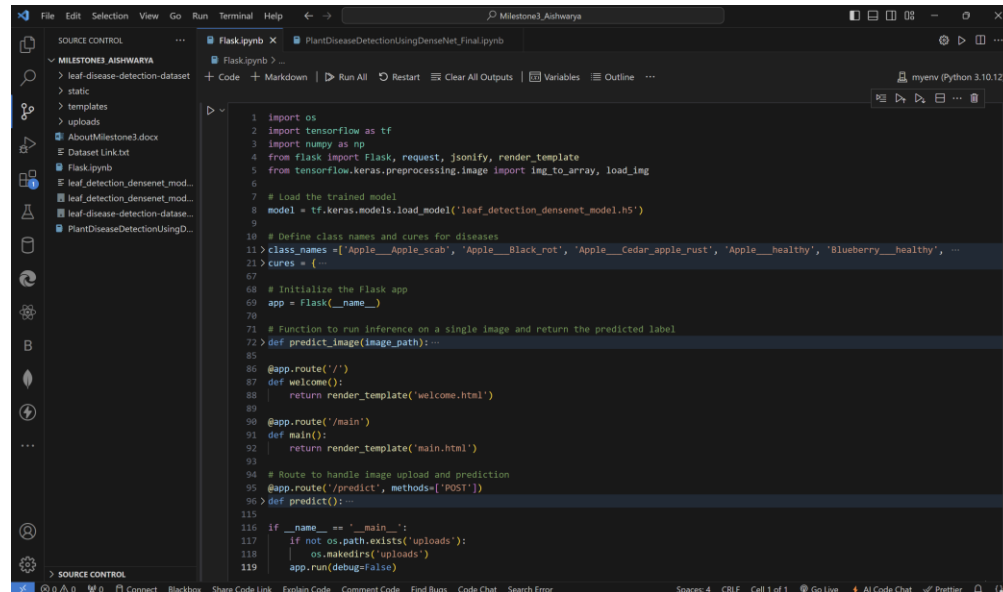
Image Pre-Processing

A total of 153 images were processed, with 150 valid images and 3 non-image files being skipped. The average dimensions of the valid images were calculated to be 256x255 pixels, maintaining an aspect ratio of 1:1. The images were resized using the INTER_AREA interpolation method to ensure consistency for further processing. This process involved reading all the images from the dataset, determining the average width and height of the valid images, and resizing them accordingly.

```
Found 43867 images belonging to 44 classes.
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/densenet/densenet121\_weights\_tf\_dim\_ordering\_tf\_kernels\_notop.h5
29084464/29084464 [=====] - 2s 0us/step
Epoch 1/10
2741/2741 [=====] - 531s 190ms/step - loss: 0.4858 - accuracy: 0.8468 - val_loss: 0.3056 - val_accuracy: 0.8954 - lr: 0.0010
Epoch 2/10
2741/2741 [=====] - 515s 188ms/step - loss: 0.2516 - accuracy: 0.9136 - val_loss: 0.2647 - val_accuracy: 0.9165 - lr: 0.0010
Epoch 3/10
2741/2741 [=====] - 515s 188ms/step - loss: 0.2038 - accuracy: 0.9310 - val_loss: 0.1277 - val_accuracy: 0.9549 - lr: 0.0010
Epoch 4/10
2741/2741 [=====] - 513s 187ms/step - loss: 0.1690 - accuracy: 0.9418 - val_loss: 0.1318 - val_accuracy: 0.9524 - lr: 0.0010
Epoch 5/10
2741/2741 [=====] - 512s 187ms/step - loss: 0.1555 - accuracy: 0.9455 - val_loss: 0.1297 - val_accuracy: 0.9550 - lr: 0.0010
Epoch 6/10
2741/2741 [=====] - 517s 189ms/step - loss: 0.1416 - accuracy: 0.9506 - val_loss: 0.1456 - val_accuracy: 0.9521 - lr: 0.0010
Epoch 7/10
2741/2741 [=====] - 517s 188ms/step - loss: 0.0585 - accuracy: 0.9788 - val_loss: 0.0521 - val_accuracy: 0.9820 - lr: 2.0000e-04
Epoch 8/10
2741/2741 [=====] - 516s 188ms/step - loss: 0.0514 - accuracy: 0.9810 - val_loss: 0.0511 - val_accuracy: 0.9827 - lr: 2.0000e-04
Epoch 9/10
2741/2741 [=====] - 519s 189ms/step - loss: 0.0444 - accuracy: 0.9844 - val_loss: 0.0403 - val_accuracy: 0.9856 - lr: 2.0000e-04
Epoch 10/10
2741/2741 [=====] - 515s 188ms/step - loss: 0.0411 - accuracy: 0.9846 - val_loss: 0.0553 - val_accuracy: 0.9798 - lr: 2.0000e-04
...
2741/2741 [=====] - 518s 189ms/step - loss: 0.0455 - accuracy: 0.9842 - val_loss: 0.0202 - val_accuracy: 0.9928 - lr: 1.0000e-04
2742/2742 [=====] - 64s 23ms/step - loss: 0.0202 - accuracy: 0.9928
Test loss: 0.020196696743369102
Test Accuracy: 0.9927735924720764
```

Model Training

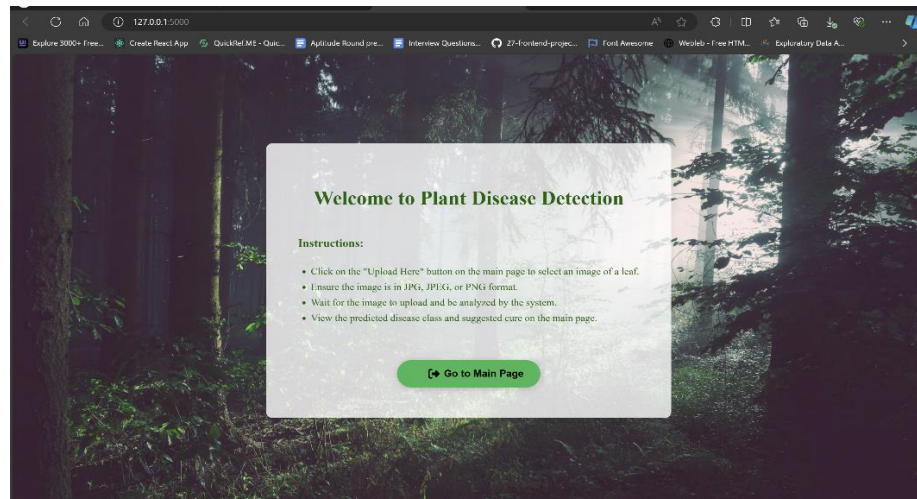
The DenseNet121 model, pre-trained on ImageNet, was fine-tuned on a dataset of 43,867 images across 44 classes over 10 epochs. Starting with a learning rate of 0.001, which was reduced to 0.0002 after the 6th epoch, the model likely used the default optimizer and categorical cross-entropy loss function. Significant improvements were seen throughout training, with accuracy rising from 84.68% to 98.46% and validation accuracy from 89.54% to 97.98%. Validation loss consistently decreased, indicating good generalization. The model achieved a test loss of 0.0202 and an impressive test accuracy of 99.28%, demonstrating excellent performance on unseen data.

A screenshot of a code editor window titled 'Milestone3_Aishwarya'. The editor shows a Python file named 'Flask.py' with the following code:

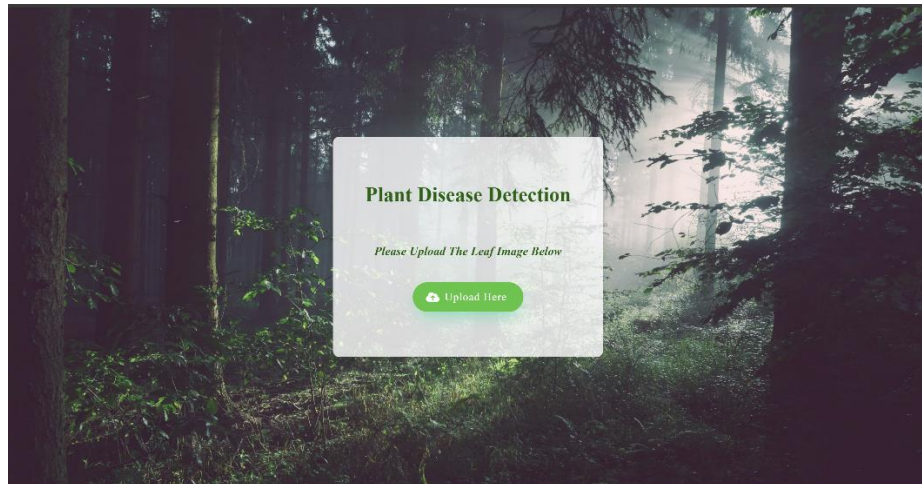
```
1 import os
2 import tensorflow as tf
3 import numpy as np
4 from flask import Flask, request, jsonify, render_template
5 from tensorflow.keras.preprocessing.image import img_to_array, load_img
6
7 # Load the trained model
8 model = tf.keras.models.load_model('leaf_detection_densenet_model.h5')
9
10 # Define class names and cures for diseases
11 class_names = ['Apple__Apple_scab', 'Apple__Black_rot', 'Apple__Cedar_apple_rust', 'Apple__healthy', 'Blueberry__healthy', ...]
12 cures = {}
13
14 # Initialize the Flask app
15 app = Flask(__name__)
16
17 # Function to run inference on a single image and return the predicted label
18 def predict_image(image_path):
19     # ...
20
21 @app.route('/')
22 def welcome():
23     return render_template('welcome.html')
24
25 @app.route('/main')
26 def main():
27     return render_template('main.html')
28
29 # Route to handle image upload and prediction
30 @app.route('/predict', methods=['POST'])
31 def predict():
32     # ...
33
34 if __name__ == '__main__':
35     if not os.path.exists('uploads'):
36         os.makedirs('uploads')
37     app.run(debug=False)
```

Model Integration

This Flask application uses a pre-trained DenseNet121 model to predict plant diseases from images. It includes a list of 44 plant disease classes, each with corresponding treatment suggestions. The application initializes a Flask server and provides two main routes: a welcome page and a main page. The key functionality is in the '/predict' route, which handles image uploads, runs the model to predict the disease, and returns the prediction and suggested cure as JSON. Uploaded images are temporarily saved for processing and then deleted. The application ensures necessary directories are created before starting the server.



Opening Page



Input Page



Output Page

- **References:** Datasets used: 1) Link to dataset containing only tea leaf diseases - <https://www.kaggle.com/datasets/saikatdatta1994/tea-leaf-disease>
- 2) Link to dataset for rest of the plants' diseases - <https://www.kaggle.com/datasets/dev523/leaf-disease-detection-dataset/data>
- 3) Loader page icon - [Loader by gustavofusco made with CSS | Uiverse.io](#)